Reading: image layers and build caching

Objectives

- Explain that RUN, COPY and ADD instructions each add an image layer and that a changed step invalidates the build cache for every later step
- Reorder a Dockerfile so dependency files are copied and installed before application source, and predict which steps rebuild after a source-only change
- Use a .dockerignore file to keep build output, version-control folders and local secret files out of the build context

Teaching

Start with a guess

Practice, not scored.

Layers and the build cache

`RUN`, `COPY` and `ADD` change files, so each adds a layer to the image. Settings such as `ENV` or `CMD` add no files.

`docker build` reuses a stored layer, shown as `CACHED`, when the step and its copied files are unchanged.

When one step changes, its cache is invalidated and every later step runs again.

Slow, stable steps first

Source changes often; dependency lists rarely. So copy the dependency files, install, then copy the source.

A source-only change then invalidates only the source copy and later steps; the slow install stays `CACHED`.

Keep the build context small

`docker build .` sends the folder, the build context, to the builder; `COPY . .` copies it all.

A `.dockerignore` file there lists what to leave out, like `.gitignore`: build output (`dist`), installed `node_modules`, `.git` and secret files such as `.env`. Ignored files never reach the image.

Apply it: a dependency change

Find the first changed step.

Model

Front-end Dockerfile and last build (simulated)

```
FROM node:24-alpine
WORKDIR /app
COPY . .
RUN npm ci
RUN npm run build
```

Nothing built yet.

Recap

- `RUN`, `COPY` and `ADD` add layers. A changed step invalidates every later step.
- Dependency files, install, then source: a source change keeps the install cached.
- `.dockerignore` keeps output, `.git` and secrets out of the context.

Next topic: Multi-stage builds

Cheat sheet

```
FROM node:24-alpine
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci
COPY . .
RUN npm run build
```

```
# .dockerignore
node_modules
```

```
dist
.git
.env
```

Sources

Checked against these live official sources on 2026-10-13. Versions shown: Docker Engine 29 with BuildKit and the node:24-alpine image. Build output is simulated and shortened: timings and some lines are left out.

- Docker Docs: Understanding the image layers
- Docker Docs: Docker build cache
- Docker Docs: Build cache invalidation
- Docker Docs: Optimize cache usage in builds (order your layers, keep the context small)
- Docker Docs: Build context and .dockerignore files
- Dockerfile reference: RUN, COPY, ADD

Static interaction alternatives

predict

Prediction (practice, not scored): with FROM node:24-alpine, WORKDIR /app, COPY . ., RUN npm ci, RUN npm run build, you change src/App.jsx and build again. Answer: COPY . . and every step after it, including npm ci, run again. The changed file invalidates COPY . ., and every later step is rebuilt. Not only the build step; not every step (FROM and WORKDIR stay cached); not none (a copied file changed).

explore

Activity: build the Dockerfile twice. With nothing changed, every step after FROM shows CACHED. After changing src/App.jsx, COPY . . changes, so it and both later steps (npm ci and npm run build) run again: [3/5] COPY . ., [4/5] RUN npm ci, [5/5] RUN npm run build.

drag

Activity: reorder the Dockerfile. Correct order after FROM node:24-alpine and WORKDIR /app: COPY package.json package-lock.json ./, then RUN npm ci, then COPY . ., then RUN npm run build. After a source change, the build shows CACHED for the package copy and npm ci, and runs only COPY . . and npm run build.

explore

Activity: choose .dockerignore entries. Ignore node_modules (installed packages, rebuilt by npm ci), dist (build output), .git (version control) and .env (local secrets). Keep package.json, package-lock.json, src, public, index.html and vite.config.js, which the build needs.

mcq

Apply question: reordered Dockerfile (FROM, WORKDIR, COPY package.json package-lock.json ./, RUN npm ci, COPY . ., RUN npm run build); you change package.json. Answer: steps 3 to 6 run again, because the first changed step is the package copy and every later step is invalidated. Not only npm ci (later steps rebuild too); not only steps 5 and 6 (package.json is copied in step 3); not none (a copied file changed). Retry as build output: run 3/6, run 4/6, run 5/6, run 6/6.